

SOFTWARE REQUIREMENTS SPECIFICATION

LIFE IN NUMBERS

Interactive Lifestyle Insight & Visualization Web Application

Document Item	Value
Document Type	Software Requirements Specification (SRS)
Project Type	College / University Web Application
Primary Technology	HTML5, CSS3, JavaScript
Architecture	Client-side web application
Business Model	B2C (Business-to-Consumer)
Theme	Royal Blue
Version	1.0

Prepared for requirement analysis, UI/UX design, student development, testing, faculty evaluation, final presentation, and future database/API planning.

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Project Overview	3
1.3 Product Purpose	3
1.4 Intended Audience	3
1.5 Definitions and Abbreviations	3
2. Overall Description	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Characteristics	4
2.4 Operating Environment	4
2.5 Design Principle	4
3. System Scope	5
3.1 In-Scope Features	5
3.2 Out-of-Scope Features	5
4. Stakeholders and User Roles	6
4.1 Stakeholder Identification	6
5. Functional Requirements	7
6. Non-Functional Requirements	8
7. User Interface and UX Requirements	9
7.1 Navigation and Page Flow	9
7.2 Visual Design	9
7.3 Responsive Design	9
7.4 Accessibility	9
8. Data Requirements	10
8.1 Input Data	10
8.2 Derived Data	10
8.3 Storage Model	10
9. System Architecture	11
9.1 Architecture Overview	11
9.2 Suggested Project Structure	11
10. Business Rules and Calculations	12
10.1 Calculation Rules	12
11. External Interfaces	13
11.1 User Interface	13
11.2 Hardware Interface	13
11.3 Software Interface	13
11.4 API Planning	13
12. Security and Privacy	14
12.1 Privacy Requirements	14
12.2 Sensitive Information	14
12.3 Future Security	14
13. Error Handling and Validation	15
14. Testing and Acceptance Criteria	16
14.1 Testing Strategy	16
14.2 Acceptance Criteria	16
15. Future Scope	17
15.1 Product Enhancements	17

15.2 Backend and API Evolution	-	17
16. Constraints and Assumptions	-	18
16.1 Constraints	-	18
16.2 Assumptions	-	18
17. Requirements Traceability Matrix	-	19
18. Conclusion	-	20
18.1 Summary	-	20
18.2 Final Project Direction	-	20

1. Introduction

1.1 Purpose

What this section contains: Explains why the SRS document exists, what it defines, and how it will support the project lifecycle.

Life in Numbers: This Software Requirements Specification defines the functional, non-functional, data, interface, architectural, security, and testing requirements of Life in Numbers. It provides a common reference for student developers, UI/UX designers, testers, project supervisors, and future maintainers.

1.2 Project Overview

What this section contains: Provides a concise description of the proposed software and the problem or opportunity it addresses.

Life in Numbers: Life in Numbers is an interactive web application that transforms everyday habits such as sleep, screen time, exercise, reading, walking, and optional savings into meaningful personalized insights. Rather than showing raw numbers, it converts them into relatable statements, visual cards, progress indicators, animations, and storytelling.

1.3 Product Purpose

What this section contains: Explains the primary value that the software provides to its users.

Life in Numbers: The product helps users understand the cumulative effect of everyday activities over months and years. It is intended for awareness, reflection, and engagement rather than medical diagnosis, health prediction, or financial advice.

1.4 Intended Audience

What this section contains: Identifies the people who will use, develop, evaluate, test, or maintain the software.

Life in Numbers: The primary audience is students and young adults. The SRS also supports student developers, project supervisors, UI/UX designers, testers, reviewers, and future developers.

1.5 Definitions and Abbreviations

What this section contains: Defines important terminology so that all project stakeholders use consistent meanings.

Life in Numbers: Life Insight means a calculated statement derived from user inputs. Life Snapshot means the final personalized summary. Local Storage means browser-based persistent storage. MVP means Minimum Viable Product. API means Application Programming Interface.

2. Overall Description

2.1 Product Perspective

What this section contains: Explains how the proposed system fits into its technical environment and whether it depends on other systems.

Life in Numbers: Version 1.0 is a lightweight client-side web application. Core functionality does not require a backend or database. Calculation, presentation, and storage modules should remain separated so that a future API/database layer can be added without redesigning the complete interface.

2.2 Product Functions

What this section contains: Summarizes the major functions that the system is expected to perform.

Life in Numbers: The system collects lifestyle information, validates inputs, performs cumulative calculations, generates contextual insights, displays results through interactive visual components, saves the latest profile locally, and generates a final Life Snapshot.

2.3 User Characteristics

What this section contains: Describes the expected users and their level of technical knowledge.

Life in Numbers: The target users are general users, particularly students and young adults. Users should not need technical knowledge to operate the application. The interface should provide clear labels, guidance, familiar controls, and understandable feedback.

2.4 Operating Environment

What this section contains: Defines the hardware and software environment in which the application is expected to run.

Life in Numbers: The application shall run in modern desktop and mobile browsers supporting HTML5, CSS3, JavaScript ES6+, and Local Storage. Current major browsers such as Chrome, Edge, Firefox, and Safari should be supported.

2.5 Design Principle

What this section contains: Defines the boundary between the academic MVP and future enhancements.

Life in Numbers: The project should remain realistic for a student development team. Core features must work without third-party APIs. Advanced services such as authentication, cloud storage, or AI are considered future enhancements rather than MVP requirements.

3. System Scope

3.1 In-Scope Features

What this section contains: Defines the features that belong to the first implementation and demonstration.

Life in Numbers: The MVP includes lifestyle input forms, input validation, calculations, personalized insight cards, progress indicators, lightweight animations, storytelling, Local Storage persistence, Life Snapshot generation, and responsive design.

3.2 Out-of-Scope Features

What this section contains: Identifies functionality deliberately excluded so the project remains manageable and focused.

Life in Numbers: The MVP excludes medical diagnosis, bank-account integration, automatic phone-data collection, mandatory cloud accounts, complex administration, advanced machine learning, and other features that would add unnecessary backend complexity.

In Scope	Out of Scope for Version 1.0
Lifestyle inputs and calculations	Medical diagnosis or treatment advice
Interactive cards and storytelling	Bank account integration
Local Storage persistence	Mandatory cloud synchronization
Life Snapshot	Advanced predictive machine learning
Responsive web interface	Complex backend administration

4. Stakeholders and User Roles

4.1 Stakeholder Identification

What this section contains: Identifies the people or groups that have an interest in the project and describes their responsibilities.

Life in Numbers: The main stakeholders are end users, the student development team, project supervisor/faculty, UI/UX designer, testers, and possible future backend developers.

Stakeholder / Role	Responsibility or Interest
End User	Enters lifestyle information and explores personalized insights.
Student Development Team	Implements interface, calculations, validation, storage, testing, and documentation.
Project Supervisor / Faculty	Reviews requirements, usability, technical implementation, and academic outcome.
UI/UX Designer	Defines layout, visual hierarchy, accessibility, responsiveness, and interaction patterns.
Tester	Verifies functional behavior, calculations, usability, compatibility, and error handling.
Future Backend Developer	May integrate authentication, database, APIs, and synchronization in later versions.

5. Functional Requirements

What this section contains: A complete, testable list of what the system must do. Each requirement is given an ID and priority so it can be mapped to implementation and testing.

ID	Requirement	Priority
FR-01	The landing page shall explain the purpose of Life in Numbers and provide a clear action to begin.	High
FR-02	The system shall collect selected lifestyle inputs through a structured form.	High
FR-03	The system shall validate required fields, numeric ranges, and invalid values before calculation.	High
FR-04	The system shall calculate cumulative values using documented formulas.	High
FR-05	The system shall generate human-readable personalized insights.	High
FR-06	The system shall update results dynamically without requiring a full page reload.	High
FR-07	The system shall display results using cards, progress indicators, icons, and lightweight animations.	Medium
FR-08	The system shall allow users to edit inputs and regenerate results.	High
FR-09	The system shall save the latest profile using Local Storage.	Medium
FR-10	The system shall restore saved data when the user returns to the same browser.	Medium
FR-11	The system shall generate a final Life Snapshot summarizing key insights.	High
FR-12	The system shall provide a reset/clear function for stored profile data.	Medium
FR-13	The system shall display methodology or source notes for predefined factual constants.	Medium
FR-14	The system shall provide responsive behavior across supported screen sizes.	High

6. Non-Functional Requirements

What this section contains: Defines how well the application should perform and the quality standards it should satisfy, including usability, performance, accessibility, reliability, privacy, and maintainability.

Category	Requirement
Usability	A first-time user should understand the main journey without external instructions.
Performance	Normal interactions and calculations should feel immediate on a typical student device.
Responsiveness	The interface shall adapt to mobile, tablet, and desktop widths without horizontal scrolling.
Accessibility	Inputs shall have visible labels, readable contrast, keyboard access, logical focus order, and understandable error messages.
Reliability	Invalid inputs shall not crash the application or produce broken visual output.
Maintainability	HTML, CSS, JavaScript, constants, storage, and calculation logic should be organized into logical modules.
Compatibility	The application should support current major browsers.
Privacy	MVP lifestyle data should remain local and should not be transmitted to a server.
Scalability	The module structure should permit future API/database integration.
Visual Consistency	Royal blue should be used consistently for primary actions, headings, active states, and selected accents.

7. User Interface and UX Requirements

7.1 Navigation and Page Flow

What this section contains: Defines how users move through the application and how the primary journey should be structured.

Life in Numbers: The recommended flow is Landing Page → Instructions/About → User Input → Life Insights → Life Snapshot. Reset and saved-data controls should remain easy to find without distracting from the primary journey.

7.2 Visual Design

What this section contains: Defines the visual language and design consistency expected across the application.

Life in Numbers: Royal blue should be used for primary buttons, headings, active states, progress elements, and selected accents. White or light neutral backgrounds should preserve readability. Components should use consistent spacing, typography, borders, shadows, and interaction states.

7.3 Responsive Design

What this section contains: Explains how the interface should adapt to different device sizes.

Life in Numbers: Desktop layouts may use multi-column cards and forms, while mobile layouts should stack content naturally. Controls should remain touch-friendly and readable without zooming.

7.4 Accessibility

What this section contains: Defines minimum accessibility expectations for forms and interactive content.

Life in Numbers: Labels should be associated with inputs, keyboard navigation should work, focus states should be visible, and errors should not rely only on color. Images and icons should have meaningful alternatives when necessary.

8. Data Requirements

8.1 Input Data

What this section contains: Defines the information that users provide to the application.

Life in Numbers: The MVP may collect age, average sleep hours per day, screen-time hours per day, exercise days per week, books read per year, steps or walking distance per day, coffee/tea consumption, study/work hours, and optional savings information.

8.2 Derived Data

What this section contains: Defines values calculated by the application rather than directly entered by the user.

Life in Numbers: Derived values may include cumulative hours, days, or years; estimated walking distance; lifetime reading count; exercise sessions; optional savings-goal estimates; and contextual comparisons.

8.3 Storage Model

What this section contains: Explains how data is stored in the MVP and how the same logical model can evolve later.

Life in Numbers: Version 1.0 may store one profile object in browser Local Storage. A future version can migrate the same logical structure into a relational or document database without changing the core user experience.

Entity	Representative Attributes
UserProfile	age, sleepHours, screenHours, exerciseDays, booksPerYear, stepsPerDay, coffeePerDay, studyWorkHours, monthlySavings(optional)
Insight	category, title, calculatedValue, unit, message, comparison(optional), source(optional)
Snapshot	generatedAt, profileSummary, selectedInsights, achievementLabel(optional)
AppSettings	animationPreference, theme, dataConsent/status(optional)

9. System Architecture

9.1 Architecture Overview

What this section contains: Explains the technical layers used to organize the application and keep the implementation maintainable.

Life in Numbers: The MVP should use a modular client-side architecture. The Presentation Layer handles HTML/CSS; the Interaction Layer handles events and animations; the Business Logic Layer handles validation, calculations, and insight generation; and the Data Layer handles Local Storage and static constants. A future Service Layer can handle REST APIs.

9.2 Suggested Project Structure

What this section contains: Provides a practical file organization for student implementation.

Life in Numbers: A suggested structure is: index.html, input.html, insights.html, snapshot.html; css/style.css; js/app.js, validation.js, calculations.js, insights.js, storage.js; data/constants.json; and assets/. The team may adapt this structure while preserving separation of responsibilities.

10. Business Rules and Calculations

10.1 Calculation Rules

What this section contains: Defines how raw user inputs are converted into derived values and ensures that calculations remain transparent and testable.

Life in Numbers: The MVP should use deterministic, documented formulas. Every assumption used in a comparison should be clearly identified in the methodology section of the application.

Rule	Example Logic
Sleep accumulation	$\text{sleepYears} = (\text{age} \times 365 \times \text{sleepHours}) / (24 \times 365)$
Screen-time accumulation	$\text{screenYears} = (\text{age} \times 365 \times \text{screenHours}) / (24 \times 365)$
Exercise sessions	$\text{exerciseSessions} = \text{age} \times 52 \times \text{exerciseDaysPerWeek}$
Books read	$\text{booksLifetime} = \text{age} \times \text{booksPerYear}$
Steps	$\text{stepsLifetime} = \text{age} \times 365 \times \text{stepsPerDay}$
Walking distance	$\text{distance} = \text{stepsLifetime} \times \text{averageStepLength}$; the assumed step length must be disclosed.
Savings goal	$\text{monthsToGoal} = \text{remainingGoal} / \text{monthlySavings}$ when $\text{monthlySavings} > 0$.
Comparison	A calculated value may be compared with a predefined factual constant, with methodology/source displayed.

Important: These calculations are estimates for awareness and engagement. They shall not be presented as medical diagnosis, health prediction, or financial advice.

11. External Interfaces

11.1 User Interface

What this section contains: Describes the primary interface through which the user interacts with the system.

Life in Numbers: The browser is the primary interface. Forms, buttons, cards, progress indicators, dialogs, navigation controls, and animated feedback shall provide all core functionality.

11.2 Hardware Interface

What this section contains: Defines whether specialized hardware is required.

Life in Numbers: No specialized hardware is required. The application should work on standard laptops, desktop computers, tablets, and smartphones.

11.3 Software Interface

What this section contains: Defines the software dependencies and compatibility expectations.

Life in Numbers: Version 1.0 requires a modern browser with JavaScript and Local Storage support. Optional future integrations may expose REST APIs.

11.4 API Planning

What this section contains: Explains how external APIs should be handled if introduced in future phases.

Life in Numbers: APIs are not required for the MVP. If used in future versions, API communication should be isolated in a service module, validated before presentation, and protected through appropriate security controls.

12. Security and Privacy

12.1 Privacy Requirements

What this section contains: Defines how user information should be protected in the MVP.

Life in Numbers: Lifestyle information should remain in the user's browser in Version 1.0. The application should not transmit personal inputs to external services. Users should be able to clear stored data.

12.2 Sensitive Information

What this section contains: Defines information the MVP should not request because it is unnecessary for the project's purpose.

Life in Numbers: The MVP shall not request passwords, banking credentials, medical records, precise location, or other sensitive information.

12.3 Future Security

What this section contains: Defines security measures that will become important if a backend is introduced.

Life in Numbers: A future backend will require HTTPS, authentication, authorization, secure session management, input sanitization, database access controls, appropriate logging, and a clear privacy notice.

13. Error Handling and Validation

What this section contains: Defines how the application responds to invalid input, missing data, storage problems, and calculation edge cases.

Scenario	Expected Behavior
Required field empty	Highlight the field, display a clear message, and prevent calculation.
Negative or invalid numeric input	Reject the value and explain the permitted range.
Unrealistic daily value	Display a warning and request correction rather than silently accepting it.
Local Storage unavailable	Continue in session-only mode and inform the user that data will not persist.
Calculation edge case	Prevent NaN/Infinity and display a safe value or explanatory message.
Reset requested	Confirm, clear saved application data, and return to a clean input state.

14. Testing and Acceptance Criteria

14.1 Testing Strategy

What this section contains: Defines how the development team will verify that the requirements have been correctly implemented.

Life in Numbers: The team should perform unit testing for calculation and validation functions, integration testing for the form-to-results flow, UI and responsive testing, browser compatibility testing, usability testing with representative students, privacy review, and regression testing after changes.

14.2 Acceptance Criteria

What this section contains: Defines the conditions under which the MVP can be considered successfully implemented.

Life in Numbers: The system should correctly validate inputs, calculate known test cases, update results after edits, save and restore data when Local Storage is available, reset data, remain usable across common screen sizes, provide accessible controls, and avoid transmitting lifestyle inputs.

Test Area	Acceptance Criterion
Form validation	Blank, invalid, and out-of-range values are handled correctly.
Calculations	Known test inputs produce expected results using documented formulas.
Dynamic results	Regeneration after edits shows current values with no stale output.
Persistence	Saved data restores after reload when Local Storage is available.
Reset	Stored data is removed and clean state is shown.
Responsive UI	Core pages remain usable without horizontal scrolling.
Accessibility	Keyboard navigation, labels, focus, and errors are usable.
Privacy	MVP does not transmit lifestyle inputs to servers/APIs.

15. Future Scope

15.1 Product Enhancements

What this section contains: Describes realistic features that can be added after the MVP.

Life in Numbers: Future versions may introduce optional user accounts, cloud synchronization, richer dashboards, downloadable PDF snapshots, personalized goals, achievement badges, anonymous benchmarks, progressive web app support, and selected third-party APIs.

15.2 Backend and API Evolution

What this section contains: Explains how the project can evolve technically without changing its core concept.

Life in Numbers: A future backend could provide REST APIs, authentication, historical snapshots, user preferences, and database storage. These features are deliberately excluded from the MVP to keep the academic implementation manageable.

16. Constraints and Assumptions

16.1 Constraints

What this section contains: Defines limitations that influence the design and implementation.

Life in Numbers: The project is intended for a student development team with limited time and resources. The first release should avoid unnecessary backend complexity, expensive services, specialized hardware, and advanced analytics.

16.2 Assumptions

What this section contains: Defines conditions assumed to be true when designing and evaluating the system.

Life in Numbers: Users are assumed to provide reasonably accurate information. Calculated results are estimates based on user input and documented assumptions. Version 1.0 is assumed to operate without a backend and without guaranteed internet access.

17. Requirements Traceability Matrix

What this section contains: Connects project objectives to requirements and verification methods so that important requirements can be traced from planning through testing.

Objective	Related Requirements	Validation
O1: Collect lifestyle information	FR-02, FR-03	Form and validation tests
O2: Convert habits into insights	FR-04, FR-05	Calculation unit tests
O3: Create engaging experience	FR-06, FR-07	UI and usability tests
O4: Preserve user experience	FR-08 to FR-12	Persistence and reset tests
O5: Deliver responsive web product	FR-14 and NFRs	Cross-device/browser tests
O6: Keep project safe and realistic	Privacy, security, scope constraints	Acceptance and privacy review

18. Conclusion

18.1 Summary

What this section contains: Provides the final interpretation of the requirements and confirms the intended project direction.

Life in Numbers: This SRS establishes a practical and implementation-oriented baseline for Life in Numbers. The requirements focus on a polished client-side web application combining HTML, CSS, JavaScript, transparent calculations, responsive UI, and personalized storytelling.

18.2 Final Project Direction

What this section contains: Confirms the boundaries of the MVP and its path toward future enhancement.

Life in Numbers: The MVP deliberately avoids unnecessary backend complexity while maintaining a clear path toward future database, API, authentication, and personalization features. Successful implementation should provide a professional academic project that can be demonstrated, tested, documented, and extended.

END OF SOFTWARE REQUIREMENTS SPECIFICATION